Reflected XSS Protected by CSP, with CSP Bypass

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting
Category	Client-Side Security
Security Mechanism	Content Security Policy
Attack Technique	CSP Bypass
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy

2. Description

Content Security Policy (CSP) is a browser security mechanism designed to reduce the risk of cross-site scripting by restricting the sources and methods through which scripts can execute.

In this lab, the application contains a reflected XSS vulnerability and attempts to mitigate it using CSP.

However, certain CSP configurations may contain weaknesses that allow attackers to bypass the intended restrictions.

A CSP bypass can make an existing injection vulnerability exploitable despite the presence of a security policy.

3. Root Cause

The underlying issue is an exploitable reflected input combined with an insufficiently restrictive CSP configuration.

Potential contributing factors include:

- Unsafe reflection of user-controlled input.
- Overly broad script source permissions.
- Trust placed in external script sources without sufficient validation.
- Inadequate nonce or hash management.
- Unsafe inline script execution permissions.
- Incorrect assumptions about CSP as a complete XSS defense.

4. Proof of Concept (PoC)

Authorized lab testing methodology:

1. Open the assigned lab.
2. Identify the reflected input parameter.
3. Inspect the HTTP response and locate the CSP header.
4. Analyze the configured directives, especially `script-src`.
5. Determine which script sources and execution methods are permitted.
6. Identify the weakness in the policy configuration.
7. Evaluate whether the existing reflected injection can execute despite CSP.
8. Confirm the behavior using browser developer tools and CSP violation messages.

Example CSP header for analysis:

```
None
Content-Security-Policy: script-src 'self'
```

This is an illustrative policy, not a representation of the lab's exact configuration.

5. Security Impact

Successful exploitation may result in:

- JavaScript execution despite CSP protection.
- Unauthorized actions in the victim's session.
- Manipulation of application content.
- Access to information exposed to client-side scripts.
- Potential compromise of application functionality.

6. Recommended Remediation

- Eliminate the underlying reflected XSS vulnerability.
- Use strict CSP policies based on nonces or hashes.
- Avoid unsafe-inline and unsafe-eval.

- Avoid trusting broad external script sources.
- Restrict script sources to explicitly approved resources.
- Validate and securely manage CSP nonces.
- Deploy CSP in enforcement mode.
- Monitor CSP violation reports.
- Apply context-aware output encoding.

7. Verification

1. Retest the reflected input.
2. Confirm that injected scripts cannot execute.
3. Verify that CSP is enforced rather than only reported.
4. Review script source permissions.
5. Test legitimate application scripts.
6. Confirm that no unexpected CSP violations occur.

8. References

- PortSwigger: Content Security Policy
- MDN: Content Security Policy
- OWASP CSP Cheat Sheet
- OWASP XSS Prevention Cheat Sheet

9. Disclaimer

For educational purposes and authorized security testing only.